Anti-Corruption Base Intelligence

BaseLine

Plantel: Centro Universitario de los Valles

Materia: Software Configuration Management

Nombre del profesor: *Omar Ali Zatarain Durán*

Grado: Tercer semestre

Prepared by: Hiram Gabriel Alonso Marquez

Index

Introduction Anti-Corruption Base Intelligence (AcBi) **2**
Objectives **3**
See the following documents: 3
Functional Architecture for version 001 ..... **4**
Development Methodology **4**
Functional Requirements **6**
Non-Functional Security Requirements 8

Introduction Anti-Corruption Base Intelligence (AcBi)

Version 2 Introduction: Furthermore, this document, 'BaselineV-2', provides the comprehensive technical specification for version 001 of the Anti-Corruption Base Intelligence (AcBi) project. It details the mobile application architecture. It also establishes the project objectives, the Agile (Scrum) development methodology, and lists the functional and non-functional requirements.

The “AcBi” project consists of designing and developing a cross-platform mobile application (See: Design Overview-V-2) for government (See:Contract Type-V-1 ans Resource Planning) to detect corruption patterns in the public sector. The solution is based on a local-first, offline approach, integrating five main functional modules: Data Listener (real-time capture and monitoring of public data), Network Analysis (construction of dynamic relationship graphs and risk scoring), Process Tracking (detailed monitoring of tenders and bureaucratic processes), Relational Time Lines (RTL, temporal comparison of declarations vs. captured signals), and Time Paradox (central dashboard with alerts and detection of chronological inconsistencies).

The platform incorporates functional and non-functional (See: Software Requirements Specification (SRS) V-1) extreme security requirements, including end-to-end encryption (E2EE), an immutable digital chain of custody using SHA-512 hashes and certified timestamps, operational anonymity through native Tor/VPN integration and automatic IP rotation, resistance to counterintelligence (sandbox detection, memory dump prevention, reverse honeypot, and programmable self-destruct), as well as protections against evidence tampering and reverse analysis.

The application is built on a Clean Architecture, with a local-first design, using Tauri 2.0 Mobile and a Rust backend for obfuscation. A key innovation is the interactive API Manager, which allows the user (detective or investigator) to manually configure data sources provided for the case (URLs, endpoints, parsing rules), visualize them in a dynamic diagram (with nodes and connections), and validate them in real time.

Limitations of the Data of Apis (LDA) for version 001:

- Everything requires manual API configuration
- New companies, changes in the Public Registry of Commerce, partners, and legal representatives.
- Opening bank accounts, financial transactions of family members or relatives, and inter-institutional transfers.
- Updates to properties, houses, land, cars, and motorcycles registered in the name of family members or close associates.
- Locations, from apps or spyware.

- Public works announcements, deadlines, awards, bids submitted, winners, budget transfers, and dates.
- Data from public projects.
- Contact information.
- Official declarative data: asset declarations, declarations of interests, public statements of officials.

Data that it does NOT support:

- Chats or personal messages
- Confidential bank details
- Any restricted data not included in the LDA

Objectives

The primary objective of AcBi is to provide a forensic intelligence tool capable of identifying sophisticated anomalies in public administration:

- Irregular Awards: Detection of anomalies in government tenders and contracts.
- Hidden Beneficiary Networks: Mapping complex relationships to find illicit connections.
- Time Paradoxes: Identifying chronological inconsistencies, such as approvals granted before official requests were filed.

See the following documents:

These documents form the basis of the present project. Consult them for more details.

- Software Requirements Specification (SRS) V-1
- Contract Type-V-1
- Design Overview-V-2
- Resource Planning-V-2

Functional Architecture for version 001

The solution integrates five specialized modules to manage the investigative workflow, see Table 1. Modules:

Module	Primary Function
Data Listener	Real-time capture and monitoring of public data from configurable APIs.
Network Analysis	Construction of dynamic relationship graphs and automated risk scoring.
Process Tracking	Detailed monitoring of bureaucratic lifecycles and tender processes.
Relational Time Lines (RTL)	Temporal comparison between official declarations and captured signals.
Time Paradox	A central dashboard for alerts and the detection of chronological anomalies.

Table 1. Modules

Development Methodology

The "AcBi" for the version 001 project. We will utilize an Agile methodology (Scrum). The system follows a Clean Architecture approach, organizing code into independent layers to facilitate maintenance:

- Development Approach It will follow a Hybrid Development model: React with JavaScript for interactive UIs like dashboards, timelines, and charts via vis.js and D3.js with Rust for logic, business, and obfuscation.
- The system uses Tauri with React Native for a multiplatform app, PC, and Android.
- The structure is Clean Architecture with Local-First (Storage). To maintain total control over sensitive data, the app runs without third-party central servers.

- FrontendUI is made by React Native with vis.js and D3.js for the dashboard, visualizations, and dynamic diagrams.

- Backend will be made with Rust for main modules, API management using Go to make many requests at the same time.

- The Encrypted Database is local storage using SQLCipher in the test devices.

- Comprehensive testing will be conducted for unit tests for cryptography, in order to ensure anonymity and counter-intelligence protection.

- SHA family for hashing, cryptography, VPN support, and custom Rust-based anti-forensics (Obfuscator.io for code protection), sandbox, and memory dump detection to prevent the malicious agent from obtaining a copy of the contents of RAM at a specific time.

Functional Requirements

50 Functionalities:

1. Module 1: Data Listener
    a. API Integration
    b. Verification of Manipulated APIs
    c. Metadata Capture
    d. Listening for changes.
    e. Detection of bank account openings or public financial transactions of family members.
    f. Monitoring of changes in properties and vehicles registered in the names of family members or acquaintances.
    g. API Editing: DELETE, EDIT
    h. Manipulated API Report
    i. OSINT INTEGRATION
    j. API MANAGER.

2. Module 2: Network Analysis
    a. Construction of dynamic relationship graphs (officials ↔ family members ↔ companies).
    b. Graph filtering by politician and time period.
    c. Massive data cross-referencing.
    d. Cross-referencing of projects with information from Module 1.
    e. Risk scoring by degree of closeness.
    f. Detection of "shared accounts" or transfers between close circles.
    g. Mapping of closed groups or chats where officials and beneficiaries appear in lists.
    h. Interactive visualization of graphs with zoom.
    i. Graph update button.
    j. Filters

3. Module 3: Process Tracking
    a. Tracing the lifecycle of the data collected for the case by day, in PDF table format.
    b. Tracing by name.
    c. Monitoring of unusual accelerations in bureaucratic deadlines.
    d. Historical record of each data collection by package.
    e. AUTOMATIC COMPARISON BETWEEN SUBMITTED OFFERS AND THE WINNING ONE, BY FILE NAME
    f. Tracing of beneficiaries identified in the network graph.
    g. Tracing of budget transfers between departments
    h. Tracing of approvals.

i. Preparation of information in timelines that encompass Module 3
j. Visual timeline by process with highlighted anomalies.

4. Module 4: Relational Timelines (RTL)
    a. Generation of relational timelines by politician/institution/project.
    b. Calculation of the "noise threshold" (inconsistencies between declared and observed information).
    c. Temporal comparison: official statements vs. captured signals.
    d. Detection of approvals before official requests (Basic Time Paradox).
    e. Algorithms to predict the probability of corruption based on historical noise (Time Paradox).
    f. User-configurable threshold (e.g., >25% noise = high alert).
    g. Comparative visualization of timelines (official vs. observed).
    h. Detection of unexplained "skips" in the authorization flow. Recording of specific contradictions (e.g., "politician claims to be unaware of the company, but a family member is a partner").
    i. Export of RTLs as PDFs

5. Module 5: Time Paradox
    a. Main dashboard with filters by date, politician, institution, and project.
    b. Chronological display of all RTLs with noise thresholds.
    c. Push/email/SMS alerts when the noise threshold is exceeded.
    d. Automatic flag: "Start Investigation Case" with collected evidence.
    e. Interactive risk map by state/municipality.
    f. Bulk export of reports for NGOs, journalists, or authorities.
    g. "Paradox" mode: highlights serious temporal inconsistencies in red.
    h. History of detected cases.
    i. Interactive interface with animations.
    j. Timeline with information sheets.

Non-Functional Security Requirements

Non-Functional Requirements:

1. Data Security: All collected and processed data must be encrypted at rest and in transit.
2. Forensic-Level Security: End-to-end encryption (E2EE).
3. Immutable Digital Chain of Custody: Every piece of data captured, analyzed, or exported must have a certified SHA-512 hash and timestamp.
4. Operational Anonymity of the Investigator: Native support for VPN/Tor + automatic IP rotation.
5. Counterintelligence Resistance: Automatic detection and blocking of tracking attempts.
6. Offline Availability.
7. Scalability for Large Cases.
8. Tolerance for Outages.
9. Professional and Technical Interface: Dashboard designed for analysts.
10. Protection Against Evidence Tampering: Automatic detection of attempts to edit or inject false data.
11. Sandbox Detection and Reverse Analysis: The system must self-destruct or enter "false positive" mode if it detects that it is being analyzed in a virtual or debug environment. Code obfuscation: Binary code and scripts obfuscated with multiple layers (VMProtect, Themida, or equivalent) to hinder reverse engineering.
12. Memory dumping protection: Anti-memory dump mechanisms (anti-Process Explorer, anti-Cheat Engine, anti-DumpIt).
13. Automatic key and certificate rotation: Daily generation and renewal of encryption keys and certificates.
14. Keylogger and screen recorder detection: Automatic blocking or masking of sensitive input if capture tools are detected.
15. Traffic analysis protection: Use of padding, timing randomization, and dummy traffic to hide communication patterns.
16. Programmable self-destruction.
17. Reverse honeypot mode: The system can generate false data or deceptive signals if it detects intrusion attempts.
18. Internal researcher audit.
19. API monitoring and encryption.